

<script>

Client Side Technologies

Error Handling

</script>

ITI – Dammiate Branch

Eng. Ahmed Samir

Why Do We Need Error Handling?

✗ Without Error Handling

```
let user = JSON.parse(data);  
console.log(user.name);
```

✗ Without error handling: Program crashes, user sees nothing

✓ With Error Handling

```
try {  
  let user = JSON.parse(data);  
  console.log(user.name);  
} catch(err) {  
  console.log("Invalid data");  
}
```

✓ With error handling: Program continues gracefully

try / catch

- The try block holds risky code that might fail
- The catch block runs only if an error occurs
- The error object is automatically created and passed to catch

```
try {  
  // Risky code here  
  let result = riskyFunction();  
  console.log(result);  
} catch(err) {  
  // Handle the error  
  console.log("Error caught:", err.message);  
}
```



Program continues

Resume normal flow



Jumps to catch

Handle the error



Error occurs?

Decision: error thrown?



finally

The `finally` block always executes, whether an error occurred or not. Use it for cleanup tasks.

```
try {  
  let file = openFile("data.txt");  
  processFile(file);  
} catch(err) {  
  console.log("Error:", err.message);  
} finally {  
  closeFile(file); // Always runs!  
}
```

`finally` is perfect for cleanup: closing files, releasing resources, or resetting state.

The Error Object

When an error occurs, JavaScript automatically creates an `Error` object. You can read its properties inside the `catch` block.

- **name** - The type of error (e.g., "TypeError", "ReferenceError")
- **message** - A description of what went wrong

```
try {  
  let result = undefined.toUpperCase();  
} catch(err) {  
  console.log("Error name:", err.name);    // TypeError  
  console.log("Error message:", err.message); // Cannot read property 'toUpperCase'  
}
```

throw

Use `throw` to manually create and raise your own errors. This lets you control when an error occurs and what message it contains.

```
function checkAge(age) {  
  if (age < 18) {  
    throw new Error("Must be 18 or older");  
  }  
  console.log("Access granted!");  
}  
  
try {  
  checkAge(15);  
} catch(err) {  
  console.log("Error:", err.message); // Error: Must be 18 or older  
}
```

`throw` stops execution immediately and jumps to the `catch` block. Use it to validate input and enforce business rules.

Practical Example: Form Validation

```
function validateUser(name, age) {  
  if (!name || name.trim() === "") {  
    throw new Error("Name is required");  
  }  
  if (age < 18) {  
    throw new Error("Must be 18 or older");  
  }  
  return { name, age };  
}  
  
// Example calls:  
try {  
  validateUser("Alice", 25);  
  console.log("✓ User valid");  
} catch (err) {  
  console.log("  Error:", err.message);  
}  
  
try {  
  validateUser("", 30);  
} catch (err) {  
  console.log("  Error:", err.message); // Name is required  
}
```

This function validates input and throws errors for invalid data. The try/catch blocks handle each case gracefully.

Going Deeper: Stack Unwinding

When an error is thrown inside a function, it bubbles up through the call stack until it's caught. If it's never caught, the program crashes.

```
function funcA() {  
  funcB();  
}  
  
function funcB() {  
  funcC();  
}  
  
function funcC() {  
  throw new Error("Something went wrong!");  
}  
  
try {  
  funcA();  
} catch(err) {  
  console.log("Caught:", err.message);  
}
```

The error is thrown in funcC, then bubbles up through funcB and funcA until the try/catch catches it.

funcB → funcC



Common Built-in Error Types

Error Type	Example/Description
TypeError	Raised when a variable is not the expected type <code>undefined.toUpperCase()</code>
ReferenceError	Raised when referencing a variable that doesn't exist <code>console.log(unknownVariable)</code>
RangeError	Raised when a numeric value exceeds its valid range <code>new Array(-1)</code>
SyntaxError	Raised when there's invalid JavaScript syntax <code>let x = ;</code> (incomplete statement)



```
try {  
    // Some risky code  
    let result = JSON.parse(data);  
} catch(err) {  
    if (err instanceof SyntaxError) {  
        console.log("Invalid JSON format");  
    } else if (err instanceof TypeError) {  
        console.log("Type mismatch");  
    } else {  
        console.log("Unknown error:", err.message);  
    }  
}
```

Made with **Gamma**



The finally Pitfall: return Inside finally

Be careful! A return statement inside finally will override the return value from try or catch. This is usually a bug.

```
function test() {  
  try {  
    return "Success!";  
  } finally {  
    return "Oops!"; // This overrides the try return!  
  }  
}  
  
console.log(test()); // Prints "Oops!" (not "Success!")
```



✗ AVOID: Don't use return inside finally unless you intentionally want to override the return value.

Better practice:

```
function test() {  
  try {  
    return "Success!";  
  } finally {  
    console.log("Cleanup"); // Just do cleanup, don't return  
  }  
}
```

Custom Error Classes

For larger applications, create custom Error classes that extend the built-in Error. This makes error handling more organized and meaningful.

```
class ValidationError extends Error {  
  constructor(message) {  
    super(message);  
    this.name = "ValidationError";  
  }  
}  
  
function validateEmail(email) {  
  if (!email.includes("@")) {  
    throw new ValidationError("Invalid email format");  
  }  
}  
  
try {  
  validateEmail("notanemail");  
} catch (err) {  
  if (err instanceof ValidationError) {  
    console.log("Validation failed:", err.message);  
  }  
}
```

Custom errors make your code more readable and let you handle specific error types with instanceof.

Fail Fast vs. Collect All Errors

Fail Fast - Stop on First Error

```
function validateFast(name, age) {  
  if (!name) throw new Error("Name required");  
  if (age < 18) throw new Error("Age too low");  
  return true;  
}
```

Use case: "Good for internal logic where one error stops the process."

Collect All - Report All Errors

```
function validateAll(name, age) {  
  const errors = [];  
  if (!name) errors.push("Name required");  
  if (age < 18) errors.push("Age too low");  
  if (errors.length > 0) throw new Error(errors.join(", "));  
  return true;  
}
```

Use case: "Good for user-facing forms where you want to show all problems at once."

Recap: Error Handling Keywords

Keyword	Description
try	Block containing code that might throw an error
catch	Block that runs if an error occurs in try
finally	Block that always runs, used for cleanup
throw	Manually raise an error with a custom message
error.name	The type of error (TypeError, ReferenceError, etc.)
error.message	The error description
error.stack	The call stack showing where the error occurred
instanceof	Check if an error is a specific type
Custom errors	Create your own Error classes for specific situations



What's Next?

Error handling also applies to asynchronous code!

- **Promises** - Use `.catch()` to handle promise rejections
- **async/await** - Use `try/catch` with `async` functions for cleaner error handling
- **fetch API** - Handle network errors when fetching data from servers

```
// Coming soon: async/await error handling
async function fetchUser(id) {
  try {
    const response = await fetch(`/api/users/${id}`);
    const user = await response.json();
    return user;
  } catch(err) {
    console.log("Failed to fetch user:", err.message);
  }
}
```

These advanced patterns will be covered in the next lesson. You now have the foundation to understand them!